Illustrate the behavior with some basic examples.

```
Examples: Status points required for each level
  | initialStatus | initialStatusPoints | extraPoints | finalStatus |
  | Bronze        | 0                   | 300         | Silver      |
  | Bronze        | 100                 | 200         | Silver      |
  | Silver        | 0                   | 700         | Gold        |
  | Gold          | 0                   | 1500        | Platinum    |
```

When you implement this feature, you'll use these examples to guide your implementation. You may even expand on these examples to include edge cases or boundary conditions that are important from a technical perspective but of little interest to the business. At the unit-testing level, there are several approaches you can take.

Some developers write a single unit test to illustrate the technical implementation, and let the acceptance test verify the bulk of the examples. This works fine if the tests run quickly and the implementation is identical or very similar for each example. But when you're incrementally building a solution, it can be more convenient to have a unit-test case for each example, adding a new unit test for each example as you expand your implementation to cater to the different cases.

As you saw in the previous section, you can do this easily enough by writing separate unit tests for every example. While this works fine, it can be long-winded and repetitive, and can make refactoring harder.

Many unit-testing tools support example-driven testing to some extent. For example, both JUnit and NUnit support parameterized tests, which allow you to pass a table of values into a single unit test. The following example is in JUnit.

Listing 10.1 A data-driven unit test in JUnit

```
@RunWith(Parameterized.class)
public class WhenEarningStatusLevels {

    @Parameters
    public static Collection pointsPerStatus() {
        return Arrays.asList(new Object[][]{
                {Bronze, 0,   100,  Bronze},
                {Bronze, 0,   300,  Silver},
                {Bronze, 100, 200,  Silver},
                {Silver, 0,   700,  Gold},
                {Gold,   0,   1500, Platinum}
        });
    }

    Status initialStatus, finalStatus;
    int initialPoints, earnedPoints;

    public WhenEarningStatusLevels(Status initialStatus,
                                   int initialPoints,
                                   int earnedPoints,
                                   Status finalStatus) {
        this.initialStatus = initialStatus;
        this.initialPoints = initialPoints;
        this.earnedPoints = earnedPoints;
        this.finalStatus = finalStatus;
    }
```

The test data goes here.

The test data values are stored in these fields.

The test data is passed into the unit test via the constructor.